



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!

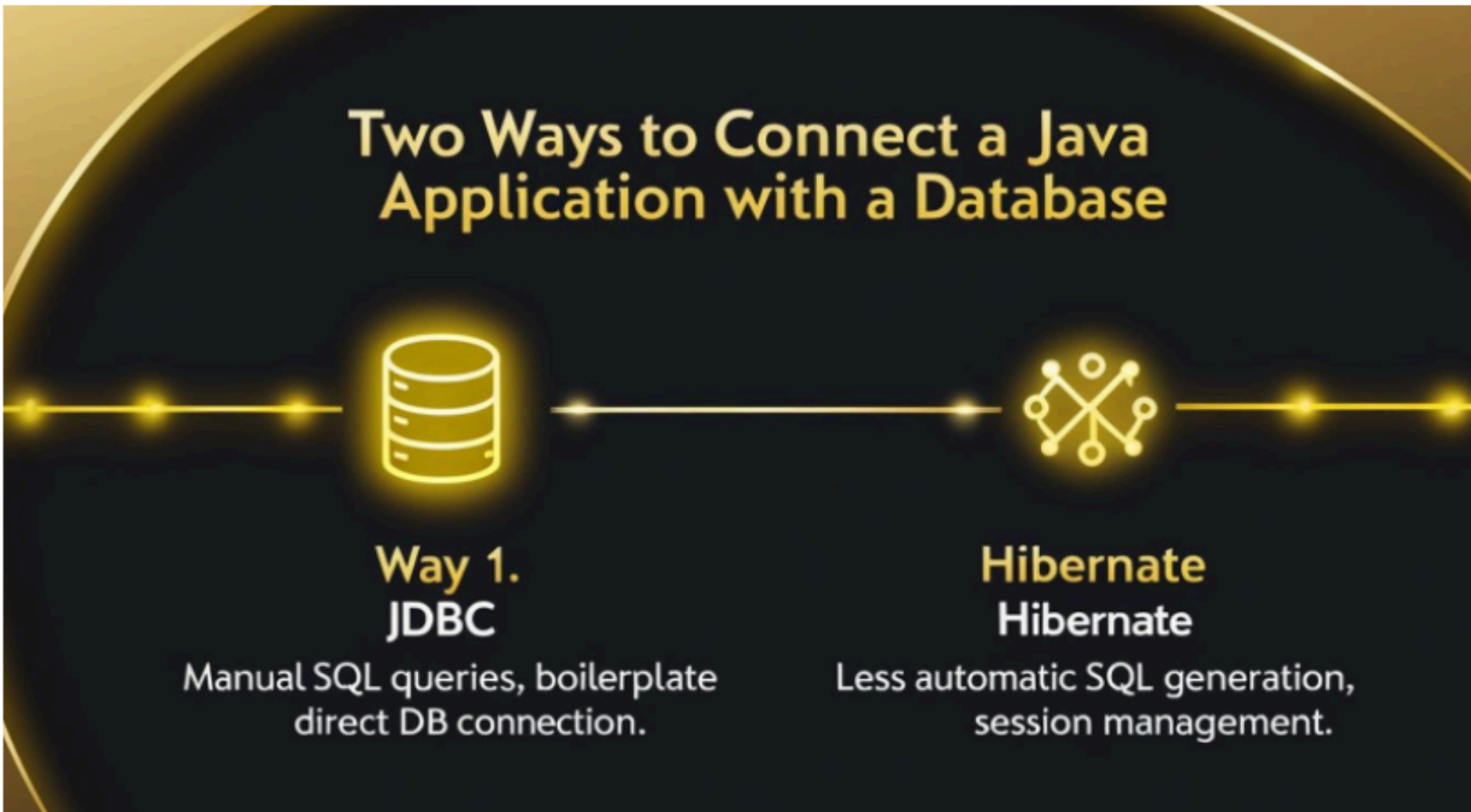


STEPS TO CONNECT OUR JAVA APPLICATION TO DATABASE USING HIBERNATE WITH EXAMPLE

How to Connect Java Spring Boot with a Database? Which one is best and why?

Mainly, there are two ways:

- Using JDBC
- Using Hibernate



- JDBC requires a lot of manual configuration, making it complex and time-consuming.
- Hibernate is the better option for establishing a database connection with a Java application.



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com

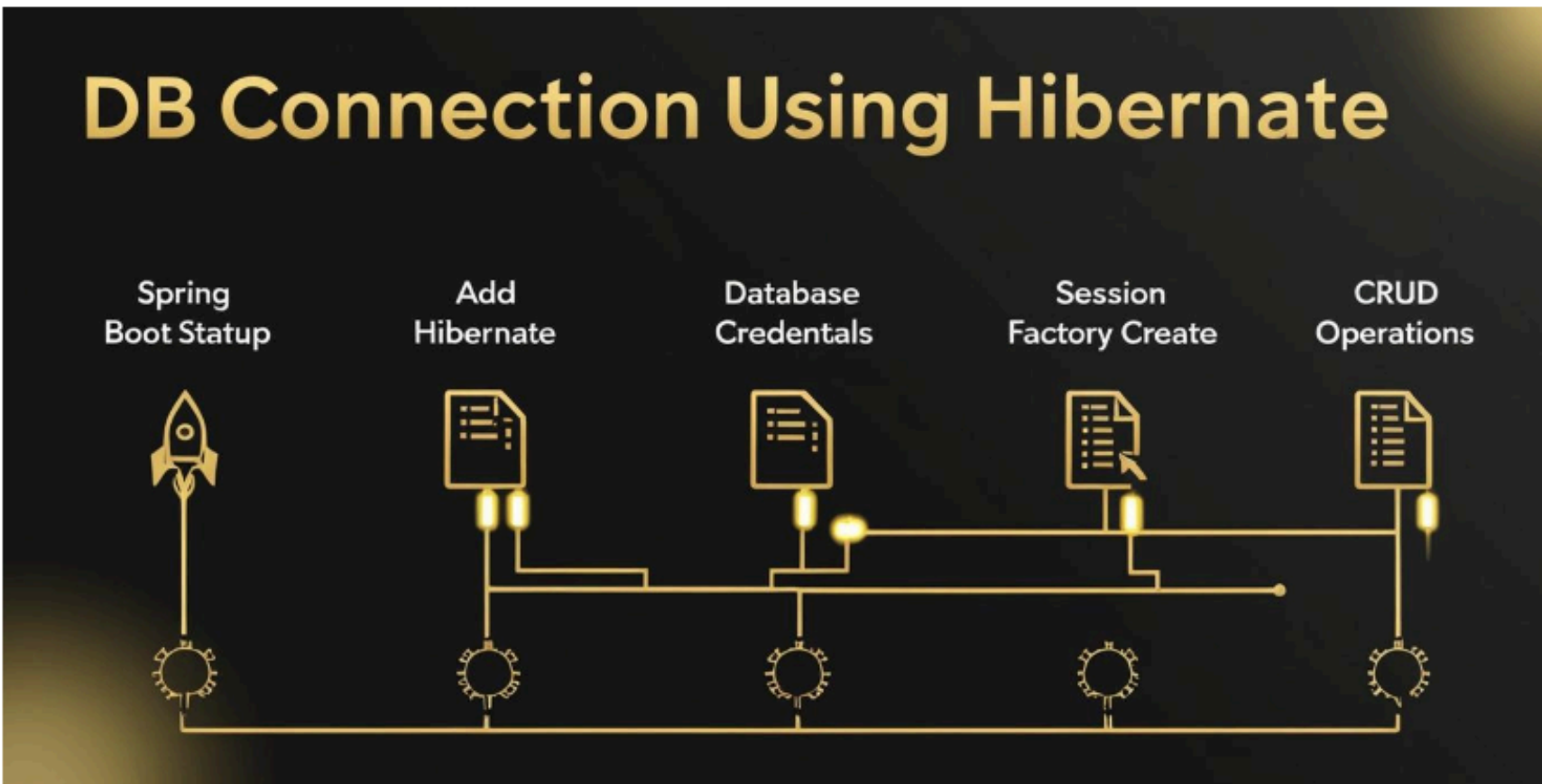


Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



STEPS TO CONNECT OUR JAVA APPLICATION TO DATABASE USING HIBERNATE WITH EXAMPLE

Steps to connect Our java application to Database using Hibernate with example



Steps to connect a Java application to a database using Hibernate:

1. Add Hibernate dependencies to your project.
2. Create a Hibernate configuration file.
3. Create entity classes to represent database tables.
4. Create a Hibernate session factory to get sessions for database operations.
5. Use Hibernate to perform database operations like creating, reading, updating, and deleting data.

These steps involve configuring your project with the necessary dependencies, defining the database connection details and settings, creating Java classes to represent the database tables, and using Hibernate to perform database operations.



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



STEPS IN DETAIL

Steps in detail

Here are the simple steps to connect a Java application to a database using Hibernate:

1. Add Hibernate dependencies to your project. You can use a build tool like Maven or Gradle to add Hibernate dependencies to your project.
 - For example, using Maven, add the following to your pom.xml:
2. Create a Hibernate configuration file. You need to create a Hibernate configuration file to define the database connection details and other Hibernate settings.
 - For example, create a file named hibernate.cfg.xml and add the following:
3. Create entity classes. You need to create Java classes to represent the database tables. These classes should have fields that match the columns in the database tables.
 - For example, create a class named User with the following fields:
4. Create a Hibernate session factory. You need to create a Hibernate session factory to get sessions for database operations.
 - For example, create a class named HibernateUtils with the following method:
5. Use Hibernate to perform database operations. You can use Hibernate to perform database operations like creating, reading, updating, and deleting data.
 - For example, to create a new user, use the following code:



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



SAMPLE CODE SNIPPET:

//Step 1: Add Hibernate dependencies to your project.

```
<dependency>
....<groupId>org.hibernate</groupId>
....<artifactId>hibernate-core</artifactId>
....<version>${hibernate.version}</version>
</dependency>
```

//Step 2: Create a Hibernate configuration file.

```
<hibernate-configuration>
  <session-factory>
....<property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
....<property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
....<property name="hibernate.connection.url">jdbc:mysql://localhost:3306/mydatabase</property>
....<property name="hibernate.connection.username">root</property>
....<property name="hibernate.connection.password">mypassword</property>
  </session-factory>
</hibernate-configuration>
```

//Step 3: Create entity classes to represent database tables.

```
@Entity
@Table(name = "users")
public class User
{
....@Id
....@GeneratedValue(strategy = GenerationType.IDENTITY)
....private Long id;

....@Column(name = "name")
....private String name;

....@Column(name = "email")
....private String email;

....// Getters and setters
}
```



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



SAMPLE CODE SNIPPET:

```
//Step 4:Create a Hibernate session factory to get sessions for database operations.
public class HibernateUtils
{
    ....private static SessionFactory sessionFactory;

    ....public static SessionFactory getSessionFactory()
    ....{
    ....    if (sessionFactory == null)
    ....    {
    ....        Configuration configuration = new Configuration().configure("hibernate.cfg.xml");
    ....        sessionFactory = configuration.buildSessionFactory();
    ....    }
    ....    return sessionFactory;
    ....}
}

//Step 5: Use Hibernate to perform database CRUD operation.

Session session = HibernateUtils.getSessionFactory().openSession();
Transaction transaction = null;
try
{
    ....transaction = session.beginTransaction();

    ....User user = new User();
    ....user.setName("John Doe");
    ....user.setEmail("john.doe@example.com");
    ....session.save(user);

    ....transaction.commit();
}
catch (Exception e)
{
    ....if (transaction != null)
    ....{
    ....    transaction.rollback();
    ....}
    ....e.printStackTrace();
}
finally
{
    ....session.close();
}
```



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
- **For more details, please perform R&D and gain a deeper understanding on each topic.**
- **Once the interview series is complete, we will cover this topic in detail.**



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com